Graph v2 Revised Implementation Plan (Decision-Complete, Codebase-Aligned)

Summary

This revision incorporates the validation feedback and aligns scope with actual repository constraints.
We keep strategy unchanged (dual-mode, event-log-first, deferred scheduler cutover), but split delivery into M1a/M1b/M1c so each milestone is implementable and verifiable.

Locked decisions:
1. Dual-mode rollout (`v1` default, `v2` flagged).
2. Event-log-first for v2 state model.
3. No scheduler replacement in M1.
4. M1 node kinds limited to `module`, `gate`, `policy`.

Scope and Milestones

M1a - Graph IR v2 + Adapters (Backend Contract Milestone)

Goal: introduce v2 model safely without runtime execution changes.

Deliverables

1. Add `GraphManifestV2Schema` and `GraphNodeV2Schema` / `GraphEdgeV2Schema`.
2. Add `GraphRunSnapshotV2Schema` (no live frontier yet, derived status only).
3. Add adapters:
   - `manifestV1ToGraphV2(manifest, recipeContext?)`
   - `graphV2ToManifestV1(graph)` (for compatibility projection)
4. Add compile output mode:
   - `--graph-mode v1`: authoritative `manifest.json`
   - `--graph-mode v2`: authoritative `graph.json` (`schemaVersion: "2.0"`) + projected `manifest.json`
   - `--graph-mode auto`: default to v1 for now
5. Persist classification metadata needed later:
   - `entryGroup` retained from split/classification pipeline (do not strip kind/category metadata).

Node kinds in M1a

- `module`
- `gate` (from recipe gates and policy-injected gate requirements)
- `policy` (policy-pack anchor nodes)

Explicit non-goals

- No runner behavior change.
- No event writer yet.
- No live frontier semantics.

M1b - API + UI v2 Read/Render (No Runner Instrumentation)

Goal: visualize v2 graph and improve operator controls while preserving latest UI baseline.

Deliverables

1. API query expansion in graph server:
   - `mode=v1|v2|auto`

   - `runId` optional (read latest by default)
2. Reader normalization:
   - Prefer `graph.json` when `mode=v2|auto` and exists.
   - Fallback to v1 run summary/artifacts.
3. UI enhancements (additive, preserve current latest design):
   - Edge-type chips (typed filtering)
   - View presets: `Overview`, `Dependencies`, `Focus`
   - Path mode: `None`, `Upstream`, `Downstream`
   - Search + `Prev/Next` navigation
   - Inspector fields: `node.kind`, `entryGroup`, `edge blocking rationale`
4. Layout policy in M1b:
   - Keep dagre.
   - No true swim lanes yet.
   - Use visual grouping (color/border/section tags) instead of spatial partitions.

Explicit non-goals

- No ELK adoption yet.
- No "running" frontier state from runtime.
- No scheduler v2.

M1c - Event Log + Snapshot Materialization (Runner Instrumentation, Still v1 Scheduler)

Goal: add reliable run telemetry substrate for future live frontier and scheduler v2.

Deliverables

1. Event file:
   - `run-summary.events.ndjson`
2. Event schemas (Zod) for:
   - `run_started`
   - `node_started`
   - `node_completed` (`success|failed|blocked|skipped`)
   - `repair_generated`
   - `critic_evaluated`
   - `budget_exhausted`
   - `run_completed`
3. Runner instrumentation insertion points in `runner.ts`:
   - At run start (after manifest load / before execution loop): `run_started`
   - Right before module command dispatch: `node_started`
   - After each module final result resolved and before write summary aggregation: `node_completed`
   - At repair task creation path: `repair_generated`
   - At budget halt decision path: `budget_exhausted`
   - At final summary write path: `run_completed`
4. Materialized snapshots:
   - `run-summary.v2.json`
   - `node-state.v2.json`
   - Derived from events by reducer utility (not computed on every API read)
5. Frontier semantics in M1c:
   - `ready`/`blocked`/`completed` accurate
   - `running` only from event stream if process active; fallback to degraded post-hoc state if not

Explicit non-goals

- No new scheduler algorithm yet.
- No lock arbitration rewrite.

M2 - Scheduler v2 MVP (Entity-Aware Execution)

Goal: add new scheduler path while reusing guardrails.

Deliverables

1. New execution path `runGraphV2(...)` behind `--graph-mode v2`.
2. Ready queue + lock arbitration using `lockKeys`.
3. Typed edge semantics (`hard/soft`, `blocking`) enforced.
4. Repair-edge injection logic for failed nodes.
5. Reuse existing:
   - critic
   - repair loop budgets
   - canary
   - test impact
   - resume logic (with digest continuity checks)

M3 - Reliability and Monorepo Scale Hardening

1. Run supervisor registry with heartbeat + cancellation tokens.
2. Strong orphan process reconciliation integrated with runId.
3. `.gitignore`-aware discovery excludes (plus existing known caches).
4. Discovery shard reuse per package/workspace segment.

M4 - Benchmark Gate and Default Switch

1. Benchmark suite compares v1 vs v2:
   - quality
   - runtime
   - token usage
   - failure recovery
   - UI observability fidelity
2. Default progression:
   - `v1` default -> `auto` default -> `v2` default after acceptance gates pass.

Public Interfaces / Types (Final)

CLI additions

1. `--graph-mode v1|v2|auto`
2. `--events-path <path>` (not `--events-file`)
3. `--run-id <id>` (for correlation and multi-run UI selection)

Stable filenames with schemaVersion

1. `graph.json` with `schemaVersion: "2.0"`
2. `run-summary.v2.json` with `schemaVersion: "2.0"`
3. `node-state.v2.json` with `schemaVersion: "2.0"`
4. `run-summary.events.ndjson` for event stream persistence

v1 <-> v2 Field Mapping (Complete, Non-lossy)

Node-level mapping

1. `DagModule.id` -> `GraphNode.id`
2. `DagModule.title` -> `GraphNode.title`
3. `DagModule.description` -> `GraphNode.description` (required in v2)
4. `DagModule.mode` -> `GraphNode.mode` (`analyze|implement` only in M1)
5. `DagModule.partPath` -> `GraphNode.execution.partPath` (required for runner projection)
6. `DagModule.command` -> `GraphNode.execution.command` (nullable)
7. `DagModule.resources` -> `GraphNode.lockKeys` (1:1 semantic carryover)
8. `DagModule.dependsOn` -> v2 edges of kind `depends_on` (authoritative); `dependsOn` in v2 is derived-only view

Run-result mapping

1. v1 result status/code/duration -> v2 node state status/telemetry
2. artifact path, critic severity, repair path retained in v2 node state extensions
3. resume digest inputs must include projected `partPath` and command text for continuity

lockKeys Specification (Final)

Definition:
- `lockKeys` identifies mutable shared scopes that must not execute concurrently.

Rules:
1. Two runnable nodes may execute in parallel only if `intersection(lockKeysA, lockKeysB) = empty`.
2. Empty `lockKeys` means unconstrained by lock system.
3. `lockKeys` derives from existing `resources` in v1 projection.
4. In M2, derivation may also include package/service ownership heuristics.

Examples:
1. Node A modifies `packages/app-shell/**` -> `lockKeys=["pkg:app-shell"]`
2. Node B modifies same package -> blocked from parallel start.
3. Node C runs tests only for `pkg:checkout` -> `lockKeys=["pkg:checkout:test"]`, parallel with app-shell changes.

entryGroup Derivation Rules (Final)

For M1:
1. Preserve compiler classification lineage:
   - preamble -> `entryGroup="core-context"`
   - stream task clusters -> `entryGroup="workstream:<slug>"`
   - postamble/gates -> `entryGroup="governance"`
2. `entryGroup` is metadata for UI grouping now; execution semantics land in M2.

Error Handling / Fallback Behavior

1. Corrupt `graph.json`:
   - API logs parse error, falls back to v1 sources, emits warning in `errors[]`.
2. Corrupt `run-summary.events.ndjson`:

   - Reducer skips malformed lines, records offsets/errors, continues.
3. v1/v2 disagreement:
   - In `mode=auto`, trust selected authoritative mode file; show mismatch warning.
4. Missing v2 artifacts in `mode=v2`:
   - Return explicit error with actionable path expectation, no silent fallback.
5. Partial write safety:
   - NDJSON writes are append with newline flush; reducer treats truncated final line as incomplete and ignores it.

Test Plan (Decision-Complete)

Unit

1. Zod validation for graph v2 schemas.
2. Event schema validation for all event types.
3. Adapter roundtrip:
   - v1 -> v2 -> projected v1 must preserve execution-critical fields.
4. lockKeys conflict checks (utility-level).
5. Reducer correctness from event sequences to snapshots.

Integration

1. CLI mode behavior:
   - `v1` writes only v1 authoritative artifacts.
   - `v2` writes v2 authoritative + v1 projection.
2. API mode resolution:
   - `v1|v2|auto` selection and fallback behavior.
3. SSE payload shape includes typed edges and graphMode.

End-to-end

1. Existing v1 workflow unchanged.
2. v2 render works with typed filters and path modes.
3. M1c event stream produces accurate materialized snapshots.
4. Resume digest continuity unaffected by v2 projection.

Acceptance Gates Per Milestone

M1a gate

1. Non-lossy mapping proven by adapter roundtrip tests.
2. Compile emits correct files per `--graph-mode`.
3. No regression in existing v1 tests.

M1b gate

1. UI controls functional on both v1-normalized and v2-native payloads.
2. Inspector shows new metadata fields correctly.
3. Dense graph interaction remains performant.

M1c gate

1. Event file created and appended safely.
2. Snapshot materialization deterministic from events.
3. Degraded frontier behavior documented and correct.

M2 gate

1. Scheduler v2 parity for guardrails and failure semantics.
2. Lock arbitration and typed blocking verified under concurrency tests.

Assumptions and Defaults

1. Default remains `--graph-mode v1` until M4 completion.
2. `analyze|implement` are the only execution modes until M2; `verify|govern` deferred.
3. v2 edges are authoritative; v2 `dependsOn` is computed compatibility view only.
4. Dagre remains layout engine in M1; ELK decision deferred pending measured need.
5. Latest graph UI stays baseline; previous-version UX patterns are additive controls only.